

CSE5095

Types and Programming Languages

Prof. Stefan Muller

Fall 2026

Programming Language =

Syntax

What programs *look like*

+

Semantics

What programs *mean*

Semantics =

Static Semantics

+

Dynamic Semantics

At compile time

Can we assign meaning to a program without running it?

Type systems

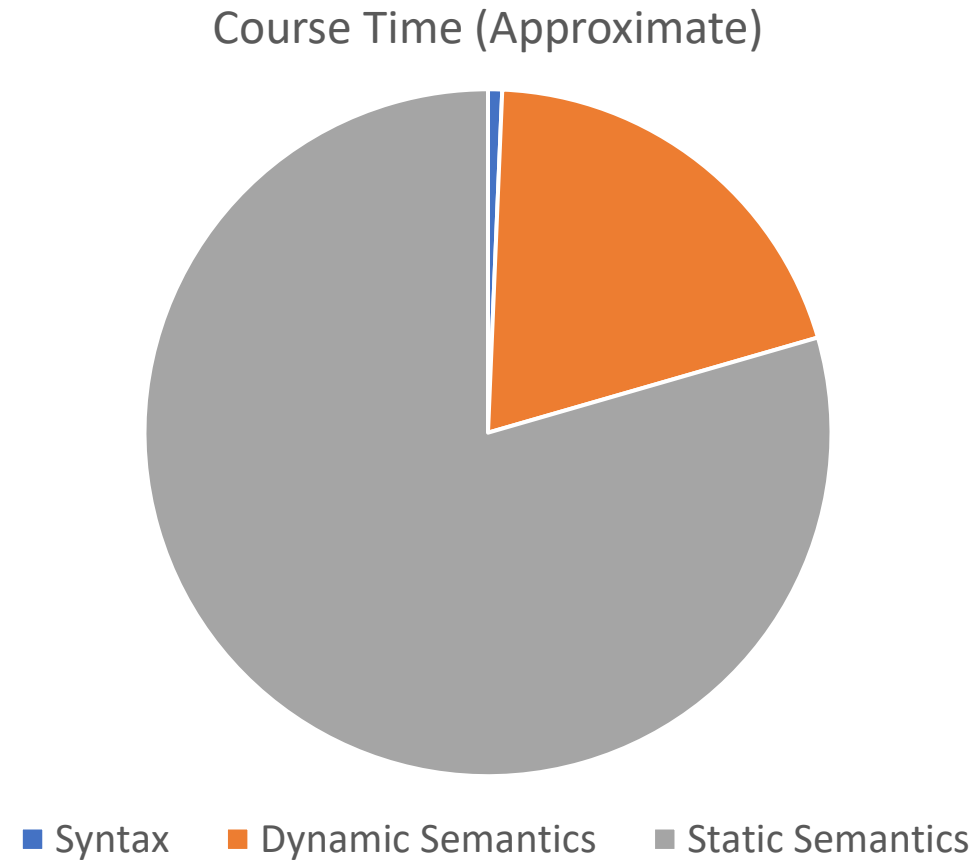
Static analysis

Verification

At run time

How do we evaluate the program?

We will not cover these topics equally



What are the semantics of this program?

`print ("Hello" + "World")`
↓
`print ("HelloWorld")`
↓
Python: HelloWorld

string int -> int -> int string

OCaml: **Error:** This expression has type string but an expression was expected of type int
Error: Unbound value print

What are the semantics of this program?

```
print (("Hello" + "World") + 42)
```



```
print ("HelloWorld" + 42)
```



Python: `TypeError: can only concatenate str (not "int") to str`

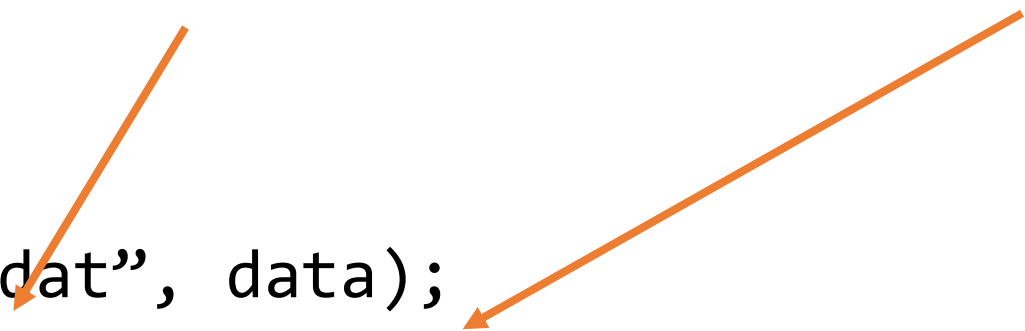
Type systems can do more than that

Does this program look right?

```
str ← input_string();           string[private]
if (string_len(str) > 10) {
    print(str);                  string[public] -> void
} else {
    print("Too short\n");
}
```

Type systems can do more than that

```
function compute (filename) {  
  file = open(filename); file isn't in the cloud!  
  ...  
}  
      (string -> unit)[server] -> unit      (string -> unit)[local]  
  
function main () {  
  ...  
  save_to_file("data.dat", data);  
  cloud_api.compute_on_cloud(compute, "data.dat");  
}
```

Two orange arrows originate from the type signatures in the first function. One arrow points from the `(string -> unit)[server]` signature to the `compute` argument in the `cloud_api.compute_on_cloud` call within the `main` function. The other arrow points from the `(string -> unit)[local]` signature to the `"data.dat"` argument in the same `cloud_api.compute_on_cloud` call.

Developing type systems is hard

- Fixing the password leak:
 - Functions that print can only take a public string
 - Fixed, right?

```
string[private] password = input_string();  
string[public] str = password;  
print(str);
```

Developing type systems is hard

- OK, let's try this again. You can't write a private string to a public string either.

```
string[private] password = input_string();
string[public] str = new string();
for (int[public] i = 0; i < MAX_PWORD_LEN; i++) {
    for (char[public] c = 'a'; c <= 'z'; c++) {
        if (c = password[i]) {
            str[i] = c;
            break;
        }
    }
}
```

Or, if this is C:

```
str = (string[public])password
```

How do we know we didn't make a mistake?

- Proofs

Theorem:

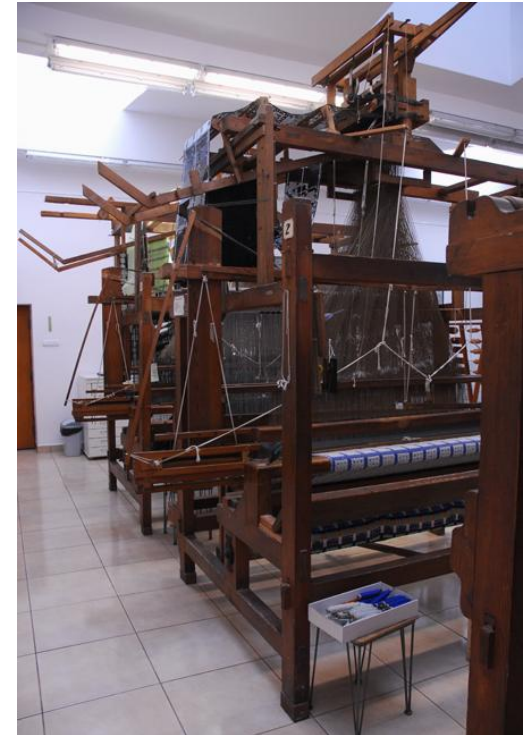
If a program P takes a private input and returns a public output
and P is well-typed
and s and s' are different private inputs,
then $P(s) = P(s')$

Proof:

(Maybe later in the semester)

“Programs” can be just about anything

“The machine was controlled by a "chain of cards"; a number of punched cards laced together into a continuous sequence.^[6] Multiple rows of holes were punched on each card...” - Wikipedia



Jacquard Loom (1804) used punch cards to indicate woven patterns

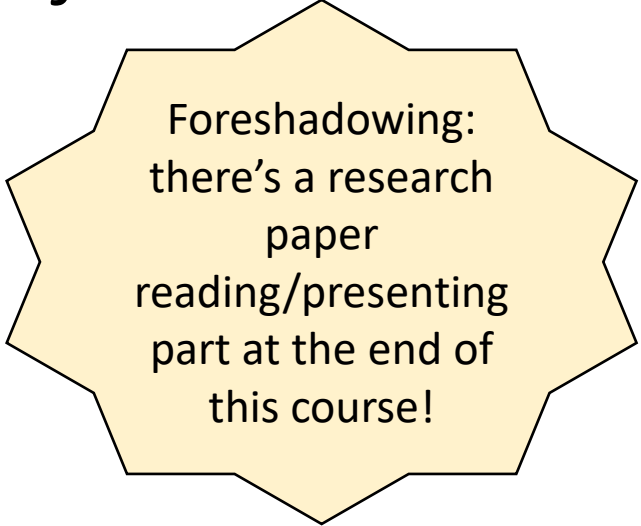
So what are the semantics of this program?



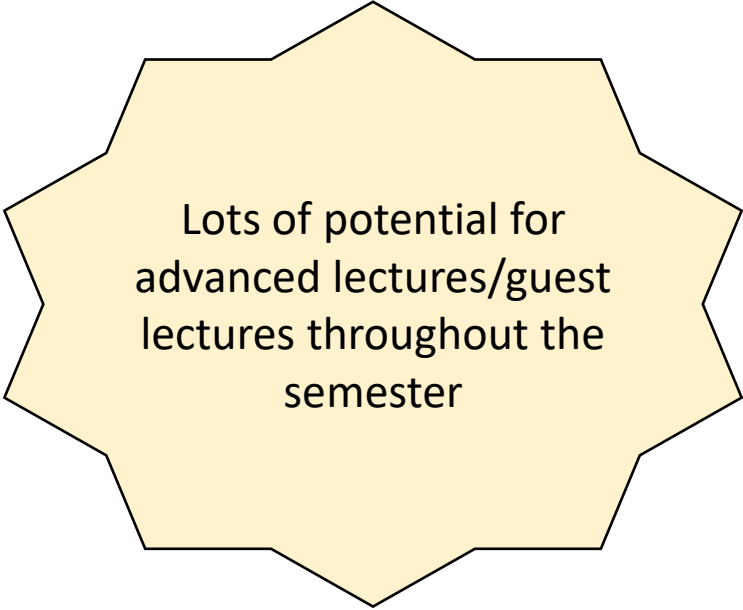
Type systems/PL techniques aren't just for programming languages

(yes, there's a PL paper for each of these)

- Distributed/Heterogenous systems
- Garbage collection
- Information security
- Memory usage/execution time
- Priority Inversions in Interactive Programs
- Modeling Unix Pipelines
- Networking
- Law
- Knitting
- Computer-aided Design (CAD)
- Music
- Roller Coasters
- Generative Interactive Systems (think text adventure games)



Foreshadowing:
there's a research
paper
reading/presenting
part at the end of
this course!



Lots of potential for
advanced lectures/guest
lectures throughout the
semester

This course (rough schedule)

- Important techniques in PL
- Modeling functional programming languages
- **Midterm (tentatively October 20)**
- Polymorphism, Parametricity and Abstraction
- Recursion and Recursive Types
- Subtyping
- Modeling imperative programming languages
- Models of Evaluation
- Advanced topics (based on time and interest)

Logistics

- HuskyCT
 - Includes syllabus and tentative schedule
 - Lecture notes will be posted there
 - Check it frequently!
- Assignments
 - Through Gradescope (access through HuskyCT)
- Discussion platform: Discord (I'll send out a link)

Prerequisites

- Familiarity with basic logic
- Comfort with mathematics, formal notations
- *Ideally: Some* programming experience in a statically typed language
- *Ideally: Some* experience programming with recursion

More about that programming prereq

- There will be no (real) programming in the class.
- Mostly theory: there **will be** proofs
 - Proofs *about* programs: we will be using some small languages
 - IMP: a small language with assignment, if/then/else, while, etc...
 - Some small functional programming languages (like Haskell, OCaml, etc.)
 - May have to write some small programs in these small languages
 - Need to be familiar with the mechanics of programming and types

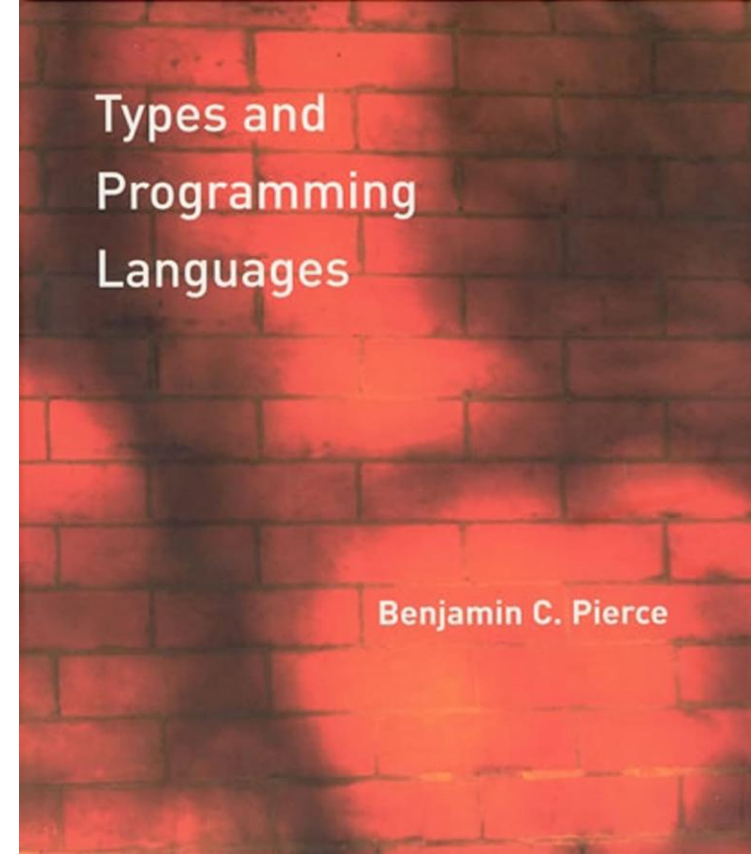


Recommended Textbook

- *Practical Foundations for Programming Languages, Second Edition* by Robert Harper (Cambridge, 2016)
- Available online as a preprint ([link on HuskyCT](#))
- Not necessary to buy it: lectures intended to be self-contained, but this is a good reference if you want one

Alternate Textbook

- *Types and Programming Languages*
by Benjamin Pierce (MIT, 2002)



Grading

- 35% Assignments
 - Roughly 6, may not be evenly weighted
- 20% Reading and presenting a research paper
 - Possibly in a group
 - More detail on this later
- 20% Midterm exam
- 25% Final exam

Exams

- Midterm: 75 minutes, normal class time
- Final exam during finals period (date set by Registrar)
 - Final exam *is* cumulative
- Some kind of notes allowed

Late Days

- 6 late days per student
- Each late day extends the deadline 24 hours
- Can use ≤ 2 per assignment
 - Can't use on exams or presentations
- No need to tell me you're using them, just submit ≤ 2 days late
- After late days used up: 10% penalty per day late
- **No work accepted >2 days late** without prior instructor approval

Submitting Homeworks

- Submit on Gradescope
- Submit a .doc, .docx, or .pdf
- Heavily preferred: Use LaTeX to generate a pdf
 - Template will be posted with assignment
 - Resources on HuskyCT

Academic Honesty Policy

- All submitted work (homework and exams) is to be your own individual work unless specified otherwise.
- Specifically prohibited (but this list isn't exhaustive):
 - Sharing answers with/getting answers from other students (even if one or both of you is done)
 - Submitting any work written by others
 - Looking online for answers
- Specifically permitted:
 - Getting help from the instructor
 - Discussing concepts or general strategies with other students
- Penalty for violation: 0 on the assignment/exam, integrity report

Academic Honesty Policy

AI

(Holds in addition to academic honesty policy from previous slide)

- All submitted work (homework and exams) is to be your own individual work unless specified otherwise.
- Specifically prohibited (but this list isn't exhaustive):
 - Sharing answers with/getting answers from ~~other students (even if one or both of you is done)~~ AI
 - Submitting any work written by ~~others~~ AI
 - ~~Looking online for answers~~
- Specifically permitted:
 - Getting help from the instructor
 - Discussing concepts or general strategies with ~~other students~~ AI
- Penalty for violation: 0 on the assignment/exam, integrity report

Asking AI

Additional note on honesty/AI policy

- Please, please contact me with any questions
- Policy is subject to change and/or clarification (with ample notice) through the semester

A bit about me

Assistant Prof at UConn since August 2025

PhD, Carnegie Mellon University 2018

2018-2020: Postdoc, Carnegie Mellon

2020-2025: Assistant Professor, Illinois Institute of Technology

Research: Type systems, language design, static analysis. *How do we make it easier to write fast, correct, secure, maintainable programs, provably?*

Office Hours

- Thursday, 1-2pm, ITE 436
 - (May change if this turns out to be a bad time for everyone)
- I'm here to answer your questions! Really! (Just not 24/7)